Учреждение образования
«Гродненский государственный политехнический колледж»
Специальность: 5-04 0612-02 «Разработка и сопровождение программного обеспечения информационных систем» (2-40 01 01 «Программное обеспечение информационных технологий»)

О Т Ч Ё Т

по лабораторной работе №20
по учебному предмету «Технология разработки программного обеспечения»
Тема: «Изучение систем контроля версий Git, GitHub, GitLab и др.»

Выполнил: И.О. Сойко
Группа: ПЗТ-45
Проверила: Е. Заяц

Гродно 2026

Цель работы

Изучить назначение и виды систем контроля версий, освоить базовые возможности Git и сервиса GitHub, научиться создавать репозиторий, работать с ветками, коммитами, удалённым репозиторием, пул-реквестами и объединением изменений на примере собственного курсового проекта.

Программное обеспечение: Git 2.56, GitHub Desktop 3.6.10, GitHub, Visual Studio 2022, браузер.

Аппаратное обеспечение: персональный компьютер.

---

1 Теоретическая часть

1.1 Системы контроля версий (CVS)

1.1.1 Назначение систем контроля версий

Система контроля версий (Version Control System, VCS; также CVS — Concurrent Versions System, историческое название класса систем) — это программное обеспечение, которое хранит историю изменений файлов и папок проекта и позволяет управлять этой историей.

Основное назначение системы контроля версий:

- фиксировать каждое изменение проекта: кто, когда и что изменил;
- хранить все предыдущие версии файлов и возвращаться к любой из них;
- сравнивать версии между собой и видеть, какие именно строки были изменены;
- обеспечивать одновременную работу нескольких разработчиков над одним проектом;
- вести параллельные линии разработки (ветки) и объединять их;
- разрешать конфликты, возникающие при изменении одних и тех же участков кода;
- создавать резервные копии и восстанавливать проект после сбоев;
- интегрироваться с системами непрерывной интеграции и доставки (CI/CD), code review, трекерами задач.

Без системы контроля версий разработчик вынужден хранить копии файлов вида «проект_финал», «проект_финал_2», «проект_точно_финал», что приводит к путанице, потере изменений и невозможности совместной работы.

1.1.2 Виды систем контроля версий

По архитектуре системы контроля версий делятся на три вида.

Локальные системы (RCS, SCCS) хранят историю изменений в специальном файле или каталоге на одном компьютере. Они подходят для небольших личных проектов, но не обеспечивают совместную работу и защиту от потери данных при поломке диска.

Централизованные системы (CVS, Subversion/SVN, Team Foundation Version Control) используют единый сервер, на котором находится хранилище с полной историей. Разработчики получают рабочие копии файлов, а изменения отправляют на сервер. Преимущество — единое хранилище и простота администрирования; недостатки — зависимость от доступности сервера и сети, а при потере сервера без резервной копии теряется вся история.

Распределённые системы (Git, Mercurial, Bazaar, Darcs) дают каждому разработчику полную копию репозитория вместе со всей историей. Локальные операции (коммит, просмотр истории, создание ветки, слияние) выполняются без обращения к серверу, а обмен изменениями происходит командами `push` и `pull`. Такие системы устойчивы к сбоям сервера, поддерживают работу офлайн и удобны для ветвления.

Вид	Примеры	Хранение истории	Совместная работа
Локальные	RCS, SCCS	На одном компьютере	Отсутствует
Централизованные	CVS, SVN, TFVC	На общем сервере	Через сервер
Распределённые	Git, Mercurial	У каждого разработчика полностью	Через обмен изменениями

1.1.3 Git: особенности подхода к работе с данными и преимущества

Git — распределённая система контроля версий, созданная Линусом Торвальдсом в 2005 году для разработки ядра Linux.

Особенности подхода Git к работе с данными:

1. **Снимки, а не разницы.** Большинство систем хранят изменения файлов в виде набора разниц (дельт). Git при каждом коммите сохраняет снимок состояния всех файлов проекта. Если файл не изменился, Git не дублирует его, а хранит ссылку на тот же объект. Это ускоряет переключение между версиями и делает историю надёжной.
2. **Адресация по хешу.** Каждый объект (файл, каталог, коммит) имеет уникальный идентификатор — SHA-1 (в новых версиях поддерживается SHA-256). Хеш вычисляется по содержимому, поэтому незаметно изменить историю невозможно: любое изменение сразу обнаруживается.

3. **Три состояния файла.** Файл в Git может находиться в рабочей области (working directory), в области подготовленных изменений (staging area, index) или в хранилище (repository, каталог `.git`). Коммит создаётся из области подготовленных изменений, что позволяет выбирать, какие именно изменения войдут в следующую версию.
4. **Лёгкие ветки.** Ветка в Git — это всего лишь указатель на коммит. Создание и переключение веток выполняются почти мгновенно и не требуют копирования файлов проекта.
5. **Локальность операций.** Практически все операции (`commit`, `log`, `diff`, `branch`, `merge`, `status`) работают с локальной копией репозитория и не требуют подключения к сети.
6. **Целостность и неизменяемость.** История коммитов образует связный граф; изменить старый коммит, не изменив все последующие, невозможно.

Преимущества Git:

- высокая скорость работы;
- распределённость и независимость от сервера;
- мощное и удобное ветвление и слияние;
- целостность данных благодаря хешированию;
- возможность работы офлайн;
- широкая распространённость и поддержка платформами GitHub, GitLab, Bitbucket;
- огромная экосистема инструментов и интеграций (CI/CD, code review, GitHub Actions).

Недостатки: сравнительно высокий порог входа, сложность некоторых команд, необходимость Git LFS для больших бинарных файлов.

1.1.4 Установка и настройка Git на ПК

1. Скачать установщик с официального сайта `https://git-scm.com/downloads` и запустить его. В составе GitHub Desktop уже поставляется Git, поэтому отдельная установка не обязательна.
2. Проверить установку командой:

```
git --version
git version 2.56.0.windows.4
```

1. Указать имя и электронную почту, которые будут записываться в коммиты:

```
git config --global user.name "Сойко Игнат Олегович"
git config --global user.email "hayflyf@gmail.com"
```

1. Настроить окончания строк для Windows и отображение кириллицы:

```
git config --global core.autocrlf true
```

```
git config --global core.quotepath false
git config --global i18n.commitEncoding utf-8
```

1. При необходимости задать редактор по умолчанию и проверить настройки:

```
git config --global core.editor "notepad"
git config --list --show-origin
```

1. Для работы с GitHub настроить аутентификацию: войти в GitHub Desktop либо создать Personal Access Token (Settings → Developer settings → Personal access tokens) и использовать Git Credential Manager. Дополнительно можно создать SSH-ключ командой `ssh-keygen -t ed25519 -C "hayflyf@gmail.com"` и добавить открытый ключ в GitHub (Settings → SSH and GPG keys).

1.2 GitHub

1.2.1 Назначение сервиса GitHub

GitHub — облачная платформа для хранения Git-репозиториев и совместной разработки. GitHub предоставляет:

- удалённое хранение репозиториев (публичных и приватных);
- совместную работу и разграничение прав доступа;
- пул-реквесты (pull requests) и код-ревью;
- трекер задач (Issues) и проекты (Projects);
- непрерывную интеграцию и доставку (GitHub Actions);
- хостинг статических сайтов (GitHub Pages);
- пакеты (GitHub Packages), релизы, вики, обсуждения;
- социальные функции: профиль разработчика, подписки, звёзды, форки.

GitHub не заменяет Git, а дополняет его: Git — это инструмент контроля версий, а GitHub — сервис для хранения и совместной работы.

1.2.2 Назначение GitHub Desktop

GitHub Desktop — официальное графическое приложение для Windows и macOS, упрощающее работу с Git и GitHub. Оно позволяет:

- клонировать репозитории и создавать новые;
- просматривать изменения файлов и формировать коммиты;
- создавать, переключать и удалять ветки;
- выполнять `push` и `pull` без командной строки;
- разрешать конфликты слияния;
- создавать пул-реквесты и переходить на страницу репозитория;
- открывать проект в Visual Studio Code или другой редактор.

GitHub Desktop удобен для начинающих, но не отменяет знание команд Git: сложные операции всё равно выполняются в консоли.

1.2.3 Основные термины Git и GitHub

1. **Репозиторий (repository)** — хранилище проекта вместе со всей историей изменений. Бывает локальным (на компьютере) и удалённым (на GitHub).
2. **Рабочая область и хранилище** — рабочая область (working directory) содержит текущие файлы проекта; область подготовленных изменений (staging area/index) — изменения, отобранные для следующего коммита; хранилище (repository, каталог `.git`) — зафиксированная история. Это три состояния файлов в Git.
3. **Форк (fork)** — личная копия чужого репозитория в своём аккаунте. Форк создаётся кнопкой **Fork** и позволяет независимо разрабатывать проект, а затем предлагать изменения в исходный репозиторий через пул-реквест.
4. **Ветка (branch)** — независимая линия разработки, указывающая на определённый коммит. Позволяет вести новую функцию или исправление, не затрагивая основную ветку.
5. **Мастер (master/main)** — основная (главная) ветка репозитория. В современных проектах по умолчанию используется имя `main`, но исторически применялось `master`.
6. **Коммит (commit)** — зафиксированное состояние проекта. Коммит хранит снимок файлов, автора, дату, сообщение и хеш, а также ссылку на родительский коммит.
7. **Пул (pull)** — получение изменений из удалённого репозитория и их слияние с локальной веткой.
8. **Пуш (push)** — отправка локальных коммитов в удалённый репозиторий.
9. **Пул-реквест (pull request, PR)** — предложение внести изменения из одной ветки или форка в основную ветку. Пул-реквест позволяет обсудить изменения, провести код-ревью и выполнить слияние.
10. **Мердж (merge)** — слияние одной ветки с другой. При изменении одних и тех же строк возникает конфликт, который разрешается вручную.
11. **Кодовое ревью (code review)** — проверка кода другими разработчиками до слияния. Позволяет найти ошибки, улучшить качество и соблюсти стандарты оформления.

1.3 GitLab и другие системы

GitLab — платформа для контроля версий и DevOps, аналогичная GitHub. Отличия: возможность установки на собственный сервер (self-hosted), встроенные CI/CD-пайплайны, merge requests вместо pull requests, развитые средства безопасности и управления проектами. GitLab удобен для организаций, которым важно хранить код внутри своей инфраструктуры.

Другие распространённые платформы:

- **Bitbucket** — интеграция с Jira и Atlassian, поддержка Git и Mercurial (ранее);
- **Azure DevOps** (ранее Team Foundation Server) — репозитории, пайплайны, доски и тестирование в экосистеме Microsoft;
- **Gitea / Gogs** — лёгкие self-hosted решения с открытым исходным кодом;
- **SourceForge** — одна из старейших площадок для открытых проектов.

Все перечисленные платформы используют Git (или совместимые системы) и поддерживают основные операции: репозиторий, ветки, коммиты, push/pull, пул-реквесты и слияние.

---

2 Практическая часть

2.1 Задание 1. Создание профиля, репозитория и размещение курсового проекта

1. Профиль на GitHub

Создан профиль GitHub: **https://github.com/HAyFli** (имя пользователя — `HAyFli`, отображаемое имя — HayFly).

2. Установка GitHub Desktop и создание репозитория

GitHub Desktop 3.6.10 установлен на компьютер. Локальный репозиторий создан на диске D: в каталоге:

```
D:\GitHub\belka-krs
```

Инициализация репозитория и первичная настройка:

```
PS D:\GitHub\belka-krs> git init -b main
Initialized empty Git repository in D:/GitHub/belka-krs/.git/

PS D:\GitHub\belka-krs> git config user.name "Сойко Игнат Олегович"
PS D:\GitHub\belka-krs> git config user.email "hayflyf@gmail.com"
PS D:\GitHub\belka-krs> git config core.quotepath false
```

3. Размещение курсового проекта в репозитории

В репозиторий помещён курсовой проект «Белочка спасает мир» — 2D-платформер на C++ с библиотекой raylib.

Файл / каталог	**Назначение**
`Source.cpp`	Исходный код игры: логика, уровни, отрисовка
`Header.h`	Вспомогательные структуры и константы

Файл / каталог	**Назначение**
`KRS.sln`	Решение Visual Studio
`KRS.vcxproj`, `KRS.vcxproj.filters`	Файл проекта и фильтры
`packages.config`	Зависимость raylib 5.5.0 (NuGet)
`image/`	Текстуры, фоны уровней, спрайты
`arial.ttf`	Шрифт с поддержкой кириллицы
`NewProject.chm`	Справка по игре (открывается по F1)
`installer/BelkaSetup_v1.0.exe`	Файл SETUP — установщик игры
`docs/ПЗ_Белочка_спасает_мир.docx`	Пояснительная записка (ПЗ)
`docs/ГЧ_Приложение_А_диаграммы.docx`	Графическая часть (ГЧ): диаграммы вариантов использования, классов и деятельности
`README.md`	Описание проекта
`.gitignore`	Исключение служебных и сборочных файлов

Всего в первый коммит вошло 30 файлов:

```
PS D:\GitHub\belka-krs> git add -A
PS D:\GitHub\belka-krs> git commit -m "Initial commit: курсовой проект
«Белочка спасает мир» (C++/raylib)"
[main (root-commit) 8a1d91e] Initial commit: курсовой проект «Белочка спасает
мир» (C++/raylib)
 30 files changed, 2192 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 Header.h
 create mode 100644 KRS.sln
 create mode 100644 KRS.vcxproj
 create mode 100644 KRS.vcxproj.filters
 create mode 100644 NewProject.chm
 create mode 100644 README.md
 create mode 100644 Source.cpp
 create mode 100644 arial.ttf
 create mode 100644 docs/ГЧ_Приложение_А_диаграммы.docx
 create mode 100644 docs/ПЗ_Белочка_спасает_мир.docx
 create mode 100644 image/block.png
 ...
```

```
 create mode 100644 installer/BelkaSetup_v1.0.exe
 create mode 100644 packages.config
```

4. Интеграция локального репозитория с GitHub

На GitHub создан публичный репозиторий `belka-krs`, после чего локальный репозиторий связан с удалённым и изменения отправлены:

```
PS D:\GitHub\belka-krs> git remote add origin
https://github.com/HAyFli/belka-krs.git
PS D:\GitHub\belka-krs> git remote -v
origin  https://github.com/HAyFli/belka-krs.git (fetch)
origin  https://github.com/HAyFli/belka-krs.git (push)

PS D:\GitHub\belka-krs> git push -u origin main
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Репозиторий доступен публично, без авторизации.

5. Ссылка на репозиторий

https://github.com/HAyFli/belka-krs

Ссылка на профиль: **https://github.com/HAyFli**

2.2 Задание 2. Типовые действия с проектом

На примере курсового проекта продемонстрированы основные действия, выполняемые при работе с Git и GitHub.

№	Действие	Команда / интерфейс	Результат
1	Создание репозитория	`git init -b main`	Создан локальный репозиторий с веткой `main`
2	Создание мастера (основной ветки)	`git branch -M main`	Основная ветка называется `main`
3	Создание ветки	`git switch -c feature/lab20-report`	Создана ветка для отчёта
4	Коммит	`git add .` и `git commit -m "..."`	Изменения зафиксированы в истории
5	Пуш	`git push -u origin feature/lab20-report`	Ветка отправлена на GitHub

№	Действие	Команда / интерфейс	Результат
6	Пул-реквест	GitHub → Compare & pull request → Create pull request	Создан PR №1
7	Мердж	Merge pull request → Confirm merge	Ветка влита в `main`
8	Пул	`git switch main` и `git pull origin main`	Изменения получены в локальный `main`
9	Форк	Кнопка **Fork** на странице чужого репозитория	Создана личная копия репозитория
10	Дерево коммитов	`git log --graph --oneline --all`	Показана история в виде графа

2.2.1 Создание ветки и коммит

Для подготовки отчёта создана отдельная ветка, чтобы не изменять основную ветку до проверки:

```
PS D:\GitHub\belka-krs> git switch -c feature/lab20-report
Switched to a new branch 'feature/lab20-report'

PS D:\GitHub\belka-krs> git status
On branch feature/lab20-report
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        LAB20_Git_Report.md

PS D:\GitHub\belka-krs> git add LAB20_Git_Report.md
PS D:\GitHub\belka-krs> git commit -m "docs: добавлен отчёт по лабораторной
работе №20"
[feature/lab20-report 2186545] docs: добавлен отчёт по лабораторной работе
№20
 1 file changed, 409 insertions(+)
```

2.2.2 Отправка ветки на GitHub и создание пул-реквеста

```
PS D:\GitHub\belka-krs> git push -u origin feature/lab20-report
 * [new branch]      feature/lab20-report -> feature/lab20-report
branch 'feature/lab20-report' set up to track 'origin/feature/lab20-report'.
```

После отправки ветки на GitHub открыт пул-реквест **№1** «docs: добавлен отчёт по лабораторной работе №20» из ветки `feature/lab20-report` в `main`:

https://github.com/HAyFli/belka-krs/pull/1

2.2.3 Слияние (мердж) пул-реквеста

В пул-реквесте нажата кнопка **Merge pull request**, затем **Confirm merge**. GitHub выполнил слияние ветки `feature/lab20-report` с `main` и создал коммит слияния `fdf208c`.

2.2.4 Получение изменений (пул)

После слияния на GitHub локальная основная ветка обновлена:

```
PS D:\GitHub\belka-krs> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

PS D:\GitHub\belka-krs> git pull origin main
From https://github.com/HAyFli/belka-krs
 * branch            main       -> FETCH_HEAD
Updating 326c744..fdf208c
Fast-forward
 LAB20_Git_Report.md | 409
++++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 409 insertions(+)
 create mode 100644 LAB20_Git_Report.md
```

2.2.5 Просмотр дерева коммитов

```
PS D:\GitHub\belka-krs> git log --graph --oneline --all --decorate
*   fdf208c (HEAD -> main, origin/main) Merge pull request #1 from
HAyFli/feature/lab20-report
|\
| * 2186545 (origin/feature/lab20-report, feature/lab20-report) docs:
добавлен отчёт по лабораторной работе №20
|/
* 326c744 docs: указан автор курсового проекта
* 8a1d91e Initial commit: курсовой проект «Белочка спасает мир» (C++/raylib)
```

2.2.6 Форк

Форк создаётся на GitHub кнопкой **Fork** на странице чужого публичного репозитория. В результате в личном аккаунте появляется копия репозитория, связанная с исходным. Изменения из форка предлагаются в исходный проект через пул-реквест. Для собственного курсового проекта форк не требуется, так как репозиторий уже принадлежит автору.

2.2.7 Просмотр файлов проекта в репозитории

```
PS D:\GitHub\belka-krs> git ls-tree -r --name-only HEAD
.gitignore
Header.h
KRS.sln
KRS.vcxproj
KRS.vcxproj.filters
LAB20_Git_Report.md
NewProject.chm
README.md
Source.cpp
arial.ttf
docs/ГЧ_Приложение_А_диаграммы.docx
```

```
docs/ПЗ_Белочка_спасает_мир.docx
image/block.png
image/dark.png
...
installer/BelkaSetup_v1.0.exe
packages.config
```

Выводы

В ходе выполнения лабораторной работы были изучены назначение и виды систем контроля версий, особенности распределённой системы Git, основные термины Git и GitHub, а также возможности сервиса GitHub и приложения GitHub Desktop.

В практической части создан публичный репозиторий `belka-krs`, в который помещён курсовой проект «Белочка спасает мир»: исходный код на C++, файлы проекта Visual Studio, ресурсы, установщик SETUP, пояснительная записка, графическая часть и справка. Локальный репозиторий интегрирован с GitHub, выполнены коммиты, создана отдельная ветка, отправленная на удалённый репозиторий, создан и объединён пул-реквест, после чего изменения получены в локальную основную ветку командой `pull`.

В результате освоены базовые операции Git: `init`, `add`, `commit`, `status`, `log`, `branch`, `switch`, `push`, `pull`, `merge`, а также работа с удалённым репозиторием и пул-реквестами на GitHub.

Список использованных источников

1. Git: официальная документация. — URL: https://git-scm.com/doc
2. Pro Git / С. Чакон, Б. Штрауб. — URL: https://git-scm.com/book/ru/v2
3. GitHub Docs. — URL: https://docs.github.com
4. GitHub Desktop. — URL: https://desktop.github.com
5. GitLab Docs. — URL: https://docs.gitlab.com
6. «Git. Базовый курс». — URL: https://gb.ru/courses/1117
7. «Работа с Git через консоль». — URL: https://htmlacademy.ru/blog/git/git-console